

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	<i>Análisis y Diseño de Algoritmos</i>				
TÍTULO DE LA PRÁCTICA:	<i>Algoritmos: Definición, características, corrección y eficiencia</i>				
NÚMERO DE PRÁCTICA:	<i>1</i>	AÑO LECTIVO:	<i>2025-B</i>	NRO. SEMESTRE:	<i>IV (cuarto)</i>
FECHA DE PRE-SENTACIÓN:	<i>11/09/2026</i>	HORA DE PRE-SENTACIÓN:	<i>16:30</i>		
INTEGRANTE(s):				NOTA:	
<i>Villafuerte Ccapira Frank Alexis</i>					
DOCENTE(s):					
<i>Mg. Edith Giovanna Cano Mamani</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Problema numérico <i>Diseñar un algoritmo que calcule el Máximo Común Divisor (MCD) de dos números enteros mediante dos funciones una que utilice el método de Euclides y otro con bucles, comparen el tiempo de ejecución entre ambos. El autor (Cormen et al., 2022) menciona que el Máximo Común Divisor de dos números, sin que ambos sean cero, es el positivo más pequeño del conjunto $ax + by : x, y \in \mathbb{Z}$. Por tanto, el programa debe operar obteniendo el valor absoluto antes de las divisiones y verificar que ambos no sean cero a la vez.</i>

Código 1: Validación previa para evitar que los valores sean ambos cero y ordenarlos para facilitar al algoritmo.

```
1 bool validate(int& a, int& b)
2 {
3     a = abs(a);
4     b = abs(b);
5     if (b > a) //Asegurar que a>b por comodidad
6     {
7         swap(a, b);
8     }
9     if (a == b && a == 0) return false; //fallo-ambos cero
10
11     return true;
12 }
```

El MCD de dos números puede calcularse por fuerza bruta, en este caso, recorriendo los valores desde el menor de ellos hasta el número uno.

Código 2: Implementación del MCD mediante Bucle.

```
1 int LoopMCD(int a, int b)
2 {
3     if (!validate(a, b))
4         return -1;
5     if (b == 0) return a;
6     int d = b; //la validacion hace que b sea el menor
7     int count = 0;
8     while(d > 1)
9     {
10         bool condition1 = a % d == 0; //obligatorio
11         bool condition2 = b % d == 0; //si el primero es true
12         count++;
13         if (condition1)
14             count++;
15         if (condition1 && condition2) //2 divisiones
16             return d;
17         else
18             d = d - 1;
19     }
20     return 1;
21 }
```

La idea principal del algoritmo de Euclides es que el MCD de dos números es igual al MCD del menor

de ellos y el módulo de éste sobre el mayor. (Brassard, 1996) muestra la forma iterativa de realizar la operación, el cual se aplicará.

Código 3: Implementación del MCD mediante el algoritmo de Euclides. Versión Iterativa.

```
1 int EuclidesMCD(int a, int b)
2 {
3     if (!validate(a, b))
4         return -1;
5     if (b == 0) return a;
6     int count = 0;
7     int r= a % b; //primera division
8     count++;
9     while(r>0)
10    {
11        a = b;
12        b = r;
13        r = a % b; //una division
14        count++;
15    }
16    return b;
17 }
```

Tiempos de ejecución

```
a:144,b:12
Euclides: 0.0008ms
Loop: 0.0003ms
Resultado: 12
=====
a:102334155,b:165580141
Euclides: 0.0012ms
Loop: 1251.36ms
Resultado: 1
=====
a:1048576,b:2097152
Euclides: 0.0004ms
Loop: 0.0003ms
Resultado: 1048576
=====
a:16,b:7
Euclides: 0.0007ms
Loop: 0.0005ms
Resultado: 1
=====
a:9,b:48
Euclides: 0.0003ms
Loop: 0.0006ms
Resultado: 3
=====
a:0,b:0
Euclides: 0.0003ms
Loop: 0.0002ms
Resultado: -1
=====
a:4,b:0
Euclides: 0.0008ms
Loop: 0.0006ms
Resultado: 4
=====
```

¿Cuántas divisiones como máximo realiza el algoritmo de Euclides para el MCD?

Según el Teorema de Lamé, el algoritmo de Euclides para dos números tendrá un número de divisiones menor o igual a cinco veces el número de dígitos decimales del menor de ambos (Rosen, 2012)

Problema de búsqueda

Implementar un algoritmo de búsqueda de cadenas dado un texto y una palabra, determinar si la palabra se encuentra dentro del texto, deben comparar este método simple con la búsqueda de palabras en una lista ordenada, analicen qué pasa si la palabra se repite varias veces.

El método simple consiste en comparar los caracteres del texto con la palabra que se quiere encontrar.

Código 4: Implementación del Búsqueda de string Fuerza Bruta.

```
1 bool SearchBrute(const std::string& text, const std::string& word)
2 {
3     int m = word.length();
4     int n = text.length();
5     for (int i = 0; i <= n - m; i++)
6     {
7         int j;
8         for (j = 0; j < m; j++)
9             if (text.at(i + j) != word.at(j))
10                break;
11         if (j == m) return true; // encontrado
12     }
13     return false;
14 }
```

El autor (Sedgewick, 2011) indica que la búsqueda binaria es eficaz porque solo revisa pocas entradas en un arreglo ordenado, por lo cual, se puede afirmar que el tiempo de búsqueda es menor y en caso de detectar palabras repetidas, se ubicarán en espacios cercanos a la primera coincidencia. Sin embargo, la implementación actual, a diferencia del método de fuerza bruta, pasará por alto subcadenas que encajen con el patrón buscado.

Código 5: Implementación del Búsqueda de string con lista ordenada.

```
1 bool SearchList(vector<string> text, string word)
2 {
3     int left = 0, right = text.size() - 1;
4     while (left <= right)
5     { // Key is in a[lo..hi] or not present.
6         int mid = left + (right - left) / 2;
7         if (word < text[mid]) right = mid - 1;
8         else if (word > text[mid]) left = mid + 1;
9         else return true;
10    }
11    return false;
12 }
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

Palabra:Lorem
List: 0.7465ms
Brute: 0.0017ms
=====
Palabra:elementum
List: 0.6472ms
Brute: 0.076ms
=====
Palabra:dictum
List: 0.6951ms
Brute: 0.0674ms

```

En ocasiones, el resultado favorece al método de fuerza bruta, esto debido a la localidad de datos; y en este caso, la caché mejora el rendimiento (Jacob, 2008). Por otro lado, la búsqueda binaria se ve perjudicada si existen fallos de predicción. *Expliquen ¿cómo crece el tiempo respecto a la longitud del texto?, ¿cómo crece respecto al tamaño de la lista?* Cuanto más crece el texto, se realizan más comparaciones con cada letra, y en el peor caso, se debe comparar con cada caracter de la palabra a encontrar, por lo que para un texto de longitud "m" y palabra de "n" el tiempo es $\mathcal{O}(m \times n)$. Por otra parte, la búsqueda en lista dependerá del algoritmo que se use, por tanto, para una búsqueda binaria el tiempo será $\log(n)$.

Problema de clasificación

Diseñar un algoritmo que ordene una lista de nombres en orden alfabético, comparen el algoritmo de ordenamiento de la biblioteca estándar `sort()`

Para esto se implementará el algoritmo insertion sort, el cual es el más intuitivo respecto al funcionamiento.

Código 6: Implementación del algoritmo InsertionSort.

```

1 void Sort(vector<string>& words)
2 {
3     int n = words.size();
4
5     for (int i = 1; i < n; i++) {
6         string actual = words[i];
7         int j = i - 1;
8         while (j >= 0 && words[j] > actual) {
9             words[j + 1] = words[j];
10            j--;
11        }
12        words[j + 1] = actual;
13    }
14 }

```

Resultados

Repetimos la prueba más de una vez y se observó que la función `sort` de la biblioteca estándar de

C++ realiza el ordenamiento en menos tiempo.

```
Insertion Sort: 0.6002ms
=====
Sort STD: 0.4172ms
=====

Insertion Sort: 0.6561ms
=====
Sort STD: 0.458ms
=====

Insertion Sort: 0.5913ms
=====
Sort STD: 0.3942ms
=====
```

¿cómo crece el número de comparaciones si hay n nombres? Para insertion sort, se requiere comparar el elemento seleccionado con todos los elementos ya ordenados, así que cuando ya existe un elemento, se realiza una comparación, hasta el n -ésimo elemento, que debe comparar $n-1$ veces en el peor de los casos. (Sedgewick, 2011)

$$\sum_{i=1}^n i$$

Problema de decisión

Crear un algoritmo que determine si un número es palíndromo

La idea detrás de este algoritmo es obtener los dígitos de los extremos y compararlos hasta que sean diferentes o la cantidad de dígitos sea igual o menor a uno.

Código 7: Implementación de Algoritmo para determinar si un número entero es palíndromo.

```
1 bool IsPalindrome(int num)
2 {
3     int pow10 = Digits(num)-1;
4     int count = 0;
5     while (pow10 > 0)
6     {
7         int num1 = num / pow(10, pow10);
8         int num2 = num % 10;
9         if (num1 != num2) return false;
10        num = (num - num1 * pow(10, pow10)) / 10;
11        pow10 -= 2;
12    }
13    return true;
14 }
```

¿Cuántas comparaciones hace el algoritmo en función del número de dígitos d ? En vista de que las comparaciones se realizan en pares, la cantidad de comparaciones en el peor de los casos, es de $d \div 2$, lo que corresponde a un tiempo $\mathcal{O}(n)$

Problema de eficiencia:

Calcular la suma de los primeros n números naturales de dos formas: con un bucle y con la fórmula $n(n+1)/2$.

Código 8: Implementación de Suma iterativa hasta valor "n".

```

1 int AddUntilN(const int& n)
2 {
3     int out = 0;
4     int count = 0;
5     for (int i = 1; i <= n; i++)
6     {
7         out += i; // suma + asignacion = 2 pasos
8         count += 1; // no contaremos la asignacion
9     }
10    std::cout << "Sumatoria clasica->N=" << n << " Pasos=" << count << std
11    ::endl;
12    return out;
13 }
```

Resultados:

```

Sumatoria clasica->N=100 Pasos=100
Sumatoria Gauss->N=100 Pasos=3
Resultado-> gauss:5050, Iterativa=5050
=====
Sumatoria clasica->N=450 Pasos=450
Sumatoria Gauss->N=450 Pasos=3
Resultado-> gauss:101475, Iterativa=101475
=====
Sumatoria clasica->N=1000 Pasos=1000
Sumatoria Gauss->N=1000 Pasos=3
Resultado-> gauss:500500, Iterativa=500500
=====
Sumatoria clasica->N=50000 Pasos=50000
Sumatoria Gauss->N=50000 Pasos=3
Resultado-> gauss:-897458648, Iterativa=1250025000
=====
Sumatoria clasica->N=2500000 Pasos=2500000
Sumatoria Gauss->N=2500000 Pasos=3
Resultado-> gauss:412542160, Iterativa=-1734941488
=====
```

Comparar la cantidad de pasos realizados en cada caso. La fórmula de Gauss usa operaciones de suma, producto y división consecutivamente, un total de 3 pasos para cualquier valor de "n", por tanto su tiempo de ejecución siempre será $\mathcal{O}(1)$ y los pasos para la suma iterativa crecen con el valor, teniendo que sumar todos los valores hasta llegar al objetivo, por tanto, tiene un tiempo de $\mathcal{O}(n)$.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

II. SOLUCIÓN DE CUESTIONARIO

■ ¿Qué diferencia existe entre la corrección parcial y total de un algoritmo?

Un algoritmo es totalmente correcto si retorna la solución correcta para una entrada específica en un tiempo finito, además el autor (Cormen et al., 2022) menciona que un algoritmo incorrecto puede ser útil si se controla la tasa de error, por tanto podríamos considerar a estos como correctos parciales, y estos requieren un tiempo infinito o no entregan una respuesta correcta.

■ ¿Qué ventajas tiene el análisis con notación asintótica frente a medir el tiempo real de ejecución?

Es más simple de representar y depende enteramente de la lógica del algoritmo, a diferencia del tiempo, en el que varios factores se ven involucrados. Sin embargo, la forma común de representarlos es muy poco informativo según (Sedgewick, 2011).

■ Explique con un ejemplo una relación de recurrencia y su solución.

Una relación de recurrencia para una secuencia a_n es una ecuación que expresa a_n en términos de uno o más de los términos anteriores de la secuencia (Rosen, 2012). (Cormen et al., 2022) ofrece un ejemplo para resolver: Demostrar que cuando $n \geq 2$ es una potencia de 2, la solución de:

$$T(n) = \begin{cases} 2 & n=2 \\ 2T(n/2) + n & n>2 \end{cases}$$

es $T(n) = n \log n$. Para este caso usaremos la forma iterativa, en el cual debemos obtener un patrón para resolver a partir del caso base.

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + n + n$$

$$T(n) = 4T(n/4) + 2n = 2^2T(n/2^2) + 2n$$

$$T(n) = 4(2T(n/8) + n/4) + 2n$$

$$T(n) = 8T(n/8) + n + 2n$$

$$T(n) = 8T(n/8) + 3n = 2^3T(n/2^3) + 3n$$

$$T(n) = 2^kT(n/2^k) + kn$$

$$2 = n/2^k \Rightarrow 2 \times 2^k = n \Rightarrow n = 2^{k+1} \Rightarrow \log_2 n = k + 1 \Rightarrow \log_2 n - 1 = k$$

$$T(n) = 2^{\log_2 n - 1}T(2) + (\log_2 n - 1)n$$

$$T(n) = 2^{\log_2 n - 1}2 + (\log_2 n - 1)n$$

$$T(n) = 2^{\log_2 n} + (n \log_2 n - n)$$

$$T(n) = n + n \log_2 n - n$$

$$T(n) = n \log_2 n \sim n \log n$$

- **¿Por qué la eficiencia es tan importante en problemas de gran escala?**

A pesar de tener, con el paso del tiempo, computadoras más rápidas se busca programas más eficientes, principalmente por motivos económicos y de resultados. Pues puede ser desalentador que con un avance de hardware, en problemas complejos no se pueda resolver un problema en tiempo prudencial con solo agregar unos cuantos datos de entrada (Brassard, 1996).

- **¿Qué características debe cumplir un buen algoritmo? Mencione al menos tres y justifique con ejemplos.**

Finitud: Debe resolver el problema en tiempo finito. Por ejemplo, en búsqueda de caminos debe evitar pasar por los mismos puntos para evitar un ciclo cerrado.

Especificidad: Las instrucciones no deben ser ambiguas. Por ejemplo, el algoritmo de MCD de Euclides especifica que el primer valor debe ser mayor al segundo y ambos no pueden ser cero a la vez.

Uso eficiente de recursos: Debe evitar un crecimiento de tiempo muy alto, recomendable igual o menor a $n \log n$. Por ejemplo, el algoritmo bubblesort tiene un tiempo cuadrático, lo cual es un crecimiento muy alto para ser aplicado.

- **¿Qué significa que un algoritmo tenga complejidad $O(1)$? Dé un ejemplo sencillo.**

Se dice que un algoritmo tiene complejidad $O(1)$ cuando resuelve un problema independientemente de la cantidad de datos de entrada. Un ejemplo podría ser el cálculo de la sumatoria de "n" primeros números naturales.

- **Si tienes un algoritmo que resuelve un problema correctamente pero tarda demasiado, ¿qué opciones tienes como programador para mejorar su eficiencia?**

Dependiendo del problema, puede ser necesario aplicar teoría de números para buscar una solución equivalente en menos pasos. Otra opción puede ser programar a bajo nivel para ganar tiempo evitando operaciones innecesarias del compilador. Además podría aprovechar características del hardware, como la caché o paralelismo.

III. CONCLUSIONES

- Aprender algoritmos es más que hacer que el código funcione. Se debe aplicar teoría para demostrar que es correcto, y para esto se requiere lógica rigurosa para evitar fallos.
- Existe más de un camino para llegar al resultado. Pero las herramientas matemáticas permiten analizar el problema para encontrar el camino más simple y predecible.
- Se requieren implementaciones especiales para algunos problemas, pues la arquitectura del hardware puede afectar soluciones efectivas sobre el papel.

RETROALIMENTACIÓN GENERAL

Github del trabajo: <https://github.com/Frank-Villafuerte/ADA-Lab1>

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

REFERENCIAS Y BIBLIOGRAFÍA

Brassard, G. (1996). *Fundamentals of algorithmics*. Prentice Hall.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms, Fourth Edition*. MIT Press.

Jacob, B. (2008). *Memory systems: cache, DRAM, disk*. Morgan Kaufmann Publishers.

Rosen, K. H. (2012). *Discrete mathematics and its applications*. McGraw-Hill.

Sedgewick, R. (2011). *Algorithms*. Addison-Wesley.